PYTHON PROGRAMMING

10 Case Studies — Complete Solutions

Problem statements • Objectives • Assumptions • Algorithms • Flowcharts • Pseudocode • Python source code • Test cases • Sample outputs • Conclusions

Syllabus scope

Computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations and unpacking.

Important note

All thresholds, fees, penalties and sample records are illustrative assumptions for an academic exercise. Replace them with institution-approved rules before real use. The sample outputs are text output examples, not screenshots captured from a running computer.

Prepared for introductory Python coursework

How to use this document

Each case study is self-contained. The flowchart is written as a text flow so it can be copied into a diagram tool or redrawn by hand. Copy the Python code into a Python 3 interpreter and run each case independently. Code intentionally avoids external libraries and advanced features beyond the requested introductory syllabus.

Syllabus coverage across the cases

Topic	Where it appears
Variables, data types, operators	All cases
Conditional statements and loops	All cases
Strings and validation	Cases 4, 9, 10
Lists and tuples / unpacking	Cases 1–4, 6, 8, 10
Nested lists	Cases 5, 7, 9
Algorithms, flowcharts, pseudocode	All cases
Pattern generation	Not required by these ten problem statements; no unrelated pattern task added

1. University Scholarship Allocation

Problem statement

Evaluate applicants using aggregate marks, minimum subject marks, attendance and family income. Rank eligible applicants and apply deterministic tie-break rules.

Objective

Use conditions, lists, sorting and tuples to identify eligible applicants and produce a ranked scholarship list.

Assumptions

• Each applicant is a tuple: (name, aggregate, subject_marks_list, attendance_percent, annual_income).

• Eligibility: aggregate >= 80, every subject >= 35, attendance >= 75%, and income <= ■250000.

• Rank by aggregate descending, then attendance descending, then income ascending, then name alphabetically.

Algorithm

1. Read applicant records.

2. Check every eligibility condition.

3. Store eligible records with ranking fields.

4. Sort by the stated tie-break rules.

5. Print eligible ranking and rejected applicants with reasons.

Flowchart (text form)

Start → Read applicants → Check marks, attendance and income → Eligible? → Yes: store record → Sort eligible records → Display ranking → Stop. If no, record rejection reason and continue.

Pseudocode

```
FOR each applicant
    eligible = aggregate >= 80 AND attendance >= 75 AND income <= 250000
    FOR each subject_mark
        IF subject_mark < 35: eligible = False
    IF eligible: add applicant to eligible_list
    ELSE: report "Not eligible"
SORT eligible_list by aggregate descending,
     attendance descending, income ascending, name ascending
DISPLAY ranked eligible_list
```

Python source code

```
applicants = [
    ("Asha", 91, [88, 92, 90], 92, 180000),
    ("Balu", 86, [80, 32, 90], 95, 120000),
    ("Charan", 91, [90, 91, 92], 88, 200000),
    ("Divya", 84, [85, 86, 82], 80, 240000),
]

eligible = []
for name, aggregate, subjects, attendance, income in applicants:
    reasons = []
    if aggregate < 80:
        reasons.append("aggregate below 80")
    if any(mark < 35 for mark in subjects):
        reasons.append("subject mark below 35")
    if attendance < 75:
        reasons.append("attendance below 75")
    if income > 250000:
        reasons.append("income above limit")

    if reasons:
        print(name, "NOT ELIGIBLE:", ", ".join(reasons))
    else:
        eligible.append((name, aggregate, attendance, income))

eligible.sort(key=lambda x: (-x[1], -x[2], x[3], x[0]))
print("\nScholarship ranking")
for rank, (name, aggregate, attendance, income) in enumerate(eligible, 1):
    print(rank, name, "aggregate =", aggregate,
          "attendance =", attendance, "income =", income)
```

Test case(s)

Input includes one applicant with a subject mark of 32 and three applicants meeting all criteria. Two eligible applicants have aggregate 91, so attendance decides their order.

Sample output (screenshot-ready text)

```
Balu NOT ELIGIBLE: subject mark below 35

Scholarship ranking
1 Asha aggregate = 91 attendance = 92 income = 180000
2 Charan aggregate = 91 attendance = 88 income = 200000
3 Divya aggregate = 84 attendance = 80 income = 240000
```

Conclusion

The program applies all eligibility rules and ranks eligible applicants consistently using the published tie-break rules.

2. Examination Result and Grace-Mark Validation

Problem statement

Process subject marks, apply pass requirements and a limited grace-mark policy, then classify the final result without allowing marks to exceed the subject maximum.

Objective

Demonstrate loops, conditionals, lists and safe mark adjustment.

Assumptions

• Each subject is represented by (subject_name, marks, maximum).

• Pass mark is 40% of maximum, rounded up to the next whole mark.

• At most 5 grace marks may be added to a subject that is short by 1–5 marks; total grace across all subjects is at most 5.

• Final marks cannot exceed the maximum. Classification: Distinction >= 75% overall; First Class >= 60%; Second Class >= 50%; otherwise Pass if all subjects pass, else Fail.

Algorithm

1. Read subject marks and maxima.

2. Compute each subject's pass mark.

3. If short by 1–5 marks and grace remains, add only the needed grace.

4. Cap adjusted marks at the maximum.

5. Check all subjects and compute overall percentage and classification.

Flowchart (text form)

Start → Read marks → For each subject compute pass mark → Apply allowed grace if eligible → Cap at maximum → All subjects passed? → Compute percentage/classification → Display result → Stop.

Pseudocode

```
grace_used = 0
FOR each subject
    pass_mark = CEILING(0.40 * maximum)
    if marks < pass_mark AND pass_mark - marks <= 5
       AND grace_used + (pass_mark - marks) <= 5:
        add needed marks; update grace_used
    marks = MIN(marks, maximum)
all_passed = every final mark >= pass_mark
percentage = total_final / total_maximum * 100
classify using percentage and all_passed
```

Python source code

```
subjects = [
    ("Python", 38, 100),
    ("Maths", 62, 100),
    ("English", 78, 100),
]
max_total_grace = 5
grace_used = 0
final_marks = []

for name, marks, maximum in subjects:
    pass_mark = (40 * maximum + 99) // 100  # ceiling of 40%
    shortfall = pass_mark - marks
    grace = 0
    if 1 <= shortfall <= 5 and grace_used + shortfall <= max_total_grace:
        grace = shortfall
        grace_used += grace
    final_mark = min(maximum, marks + grace)
    final_marks.append((name, marks, grace, final_mark, maximum, pass_mark))

all_passed = all(final >= pass_mark
                 for _, _, _, final, _, pass_mark in final_marks)
total = sum(row[3] for row in final_marks)
maximum_total = sum(row[4] for row in final_marks)
percentage = total / maximum_total * 100
```

```
print("Subject-wise result")
for name, original, grace, final, maximum, pass_mark in final_marks:
    status = "PASS" if final >= pass_mark else "FAIL"
    print(name, "original =", original, "grace =", grace,
          "final =", final, "/", maximum, status)
print("Grace used:", grace_used)
print("Percentage:", round(percentage, 2))
if not all_passed:
    classification = "FAIL"
elif percentage >= 75:
    classification = "Distinction"
elif percentage >= 60:
    classification = "First Class"
elif percentage >= 50:
    classification = "Second Class"
else:
    classification = "Pass"
print("Classification:", classification)
```

Test case(s)

Python has 38/100, Maths 62/100, English 78/100. Python receives 2 grace marks; other marks remain unchanged.

Sample output (screenshot-ready text)

```
Subject-wise result
Python original = 38 grace = 2 final = 40 / 100 PASS
Maths original = 62 grace = 0 final = 62 / 100 PASS
English original = 78 grace = 0 final = 78 / 100 PASS
Grace used: 2
Percentage: 60.0
Classification: First Class
```

Conclusion

The solution applies a bounded grace policy, prevents marks from exceeding the maximum, and classifies only after checking every subject.

3. Traffic Violation Penalty Calculator

Problem statement

Calculate fines for multiple traffic violations using vehicle category, offence, repeat status and predefined rules, while flagging disqualification conditions.

Objective

Use tuples, dictionaries are avoided, and conditionals/loops to calculate penalties from a simple rule table.

Assumptions

• Vehicle categories: two-wheeler, car, commercial.

• Base fines are fixed in the code for each offence.

• Repeat offences double the base fine.

• Driving without a valid licence or dangerous driving flags possible disqualification for review; this is an educational example, not legal advice.

Algorithm

1. Read each violation tuple.

2. Find base fine for vehicle/offence combination.

3. Double the fine if repeat offence is true.

4. Add fine to total.

5. Flag disqualification review for serious offences.

6. Print each fine, flags and total.

Flowchart (text form)

Start → Read violations → Look up base fine → Repeat offence? → Double if yes → Add to total → Serious offence? → Flag review if yes → Print summary → Stop.

Pseudocode

```
total = 0
FOR each (vehicle, offence, repeat) in violations
    base = fine_rule(vehicle, offence)
    IF repeat: fine = base * 2
    ELSE: fine = base
    total = total + fine
    IF offence is licence_missing OR dangerous_driving:
        flag disqualification_review
DISPLAY each fine, flags, total
```

Python source code

```
# Rule format: (vehicle, offence, base_fine)
rules = [
    ("two-wheeler", "helmet", 1000),
    ("car", "seatbelt", 1000),
    ("commercial", "overload", 2000),
    ("car", "dangerous_driving", 5000),
    ("two-wheeler", "no_valid_licence", 5000),
]
violations = [
    ("two-wheeler", "helmet", False),
    ("car", "seatbelt", True),
    ("car", "dangerous_driving", False),
    ("two-wheeler", "no_valid_licence", False),
]

total = 0
review_flags = []
for vehicle, offence, repeat in violations:
    base = 0
    for rule_vehicle, rule_offence, amount in rules:
        if vehicle == rule_vehicle and offence == rule_offence:
            base = amount
            break
    if base == 0:
```

```
            print("Unknown rule:", vehicle, offence)
            continue
        fine = base * 2 if repeat else base
        total += fine
        print(vehicle, offence, "repeat =", repeat, "fine =", fine)
        if offence in ("dangerous_driving", "no_valid_licence"):
            review_flags.append(offence)

print("Total penalty:", total)
if review_flags:
    print("Disqualification review required for:", ", ".join(review_flags))
else:
    print("No disqualification review flag")
```

Test case(s)

Four violations are entered; the seatbelt offence is repeated. Two serious offences trigger a review flag.

Sample output (screenshot-ready text)

```
two-wheeler helmet repeat = False fine = 1000
car seatbelt repeat = True fine = 2000
car dangerous_driving repeat = False fine = 5000
two-wheeler no_valid_licence repeat = False fine = 5000
Total penalty: 13000
Disqualification review required for: dangerous_driving, no_valid_licence
```

Conclusion

A rule table keeps the example transparent and makes repeat penalties and serious-offence flags easy to audit. Real penalties must be replaced with current official rules.

4. String-Based Examination Answer Evaluation

Problem statement

Compare each student's answer string with an answer key, count correct, incorrect and unanswered responses, apply negative marking, and report scores.

Objective

Practice strings, lists, loops and conditional scoring.

Assumptions

• Each answer is one character: A, B, C, D, or '-' for unanswered.

• Correct answer = +1 mark; incorrect answer = -0.25; unanswered = 0.

• Score is not rounded during calculation; displayed to two decimal places.

Algorithm

1. Read answer key and each student's answers.

2. Compare answers position by position.

3. Count correct, incorrect and unanswered.

4. Calculate score using the marking rule.

5. Print student-wise result.

Flowchart (text form)

Start → Read key and student answers → Compare each position → Update correct/incorrect/unanswered → Calculate score → Display result → Next student or Stop.

Pseudocode

```
FOR each student
    correct = incorrect = unanswered = 0
    FOR i from 0 to number_of_questions - 1
        IF answer is '-': unanswered += 1
        ELSE IF answer equals key[i]: correct += 1
        ELSE: incorrect += 1
    score = correct - 0.25 * incorrect
    DISPLAY counts and score
```

Python source code

```
answer_key = "ABCDAC"
students = [
    ("Ravi", "AB-DAC"),
    ("Maya", "BBCD--"),
    ("Noor", "------"),
]

for name, answers in students:
    if len(answers) != len(answer_key):
        print(name, "INVALID: answer count does not match key")
        continue
    correct = incorrect = unanswered = 0
    for i in range(len(answer_key)):
        if answers[i] == "-":
            unanswered += 1
        elif answers[i] == answer_key[i]:
            correct += 1
        else:
            incorrect += 1
    score = correct - 0.25 * incorrect
    print(name, "correct =", correct, "incorrect =", incorrect,
          "unanswered =", unanswered, "score =", format(score, ".2f"))
```

Test case(s)

Answer key is ABCDAC. Ravi submits AB-DAC, Maya submits BBCD--, and Noor leaves every answer blank.

Sample output (screenshot-ready text)

```
Ravi correct = 5 incorrect = 0 unanswered = 1 score = 5.00
```

```
Maya correct = 2 incorrect = 3 unanswered = 1 score = 1.25
Noor correct = 0 incorrect = 0 unanswered = 6 score = 0.00
```

Conclusion

String comparison gives reproducible student-wise scores and clearly separates unanswered responses from wrong answers.

5. Nested List Classroom Occupancy Analysis

Problem statement

Represent classrooms and seat positions as nested lists, allocate students while preserving reserved seats, detect duplicate assignments and calculate vacant capacity.

Objective

Practice nested lists, list traversal, membership checks and occupancy counting.

Assumptions

• A seat contains a roll number, 'R' for reserved, or None for vacant.

• Reserved seats are not allocated in this example.

• A roll number must appear at most once across all classrooms.

Algorithm

1. Create nested classroom seating lists.

2. For each requested allocation, check duplicate roll number.

3. Find the first vacant non-reserved seat.

4. Allocate the student or report no seat.

5. Count vacant seats and print each room.

Flowchart (text form)

Start → Build room matrices → Read allocation request → Duplicate roll? → Report duplicate if yes → Find vacant seat → Allocate or report full → Count vacancies → Display → Stop.

Pseudocode

```
FOR each requested roll
    IF roll already assigned: report duplicate
    ELSE search rooms and seats
         IF seat is None: assign roll and stop search
    IF no seat found: report no available seat
FOR each room: count seats equal to None
DISPLAY room matrices and vacant capacity
```

Python source code

```
rooms = [
    [["R", None, None], [None, None, None]],
    [[None, "R", None], [None, None, None]],
]
requests = [101, 102, 103, 101, 104, 105]

def assigned_rolls(room_list):
    rolls = []
    for room in room_list:
        for row in room:
            for seat in row:
                if seat is not None and seat != "R":
                    rolls.append(seat)
    return rolls

for roll in requests:
    existing = assigned_rolls(rooms)
    if roll in existing:
        print("Roll", roll, "DUPLICATE assignment rejected")
        continue
    placed = False
    for room in rooms:
        for row in room:
            for i in range(len(row)):
                if row[i] is None:
                    row[i] = roll
                    placed = True
                    print("Roll", roll, "allocated")
                    break
            if placed:
                break
```

```
            if placed:
                break
        if not placed:
            print("Roll", roll, "not allocated: no vacant seat")

vacant_total = 0
for room_index, room in enumerate(rooms, 1):
    vacant = 0
    for row in room:
        vacant += row.count(None)
    vacant_total += vacant
    print("Room", room_index, "layout:", room, "vacant:", vacant)
print("Total vacant seats:", vacant_total)
```

Test case(s)

Two rooms each have six positions, including two reserved seats. Requests include roll 101 twice.

Sample output (screenshot-ready text)

```
Roll 101 allocated
Roll 102 allocated
Roll 103 allocated
Roll 101 DUPLICATE assignment rejected
Roll 104 allocated
Roll 105 allocated
Room 1 layout: [['R', 101, 102], [103, 104, 105]] vacant: 0
Room 2 layout: [[None, 'R', None], [None, None, None]] vacant: 5
Total vacant seats: 5
```

Conclusion

Nested lists model room layouts directly; duplicate checks and reserved-seat handling protect the seating plan.

6. Bus Passenger Occupancy Simulation

Problem statement

Simulate passenger boarding and alighting at several stops, track passengers after each stop, and flag capacity violations.

Objective

Use loops, tuples, arithmetic and conditions to maintain a running passenger count.

Assumptions

• Each stop is a tuple: (stop_name, boarding, alighting).

• The bus starts empty and has capacity 40.

• Alighting cannot exceed the number of passengers currently on board; invalid input is reported and not applied.

• Capacity violations are flagged if passengers exceed 40.

Algorithm

1. Set passenger count to zero.

2. For each stop, validate alighting count.

3. Subtract alighting, add boarding.

4. Check for capacity violation.

5. Print stop-wise passenger count and final status.

Flowchart (text form)

Start → Set passengers = 0 → Read stop event → Validate alighting → Update passengers → Over capacity? → Flag if yes → Print stop count → Next stop → Stop.

Pseudocode

```
passengers = 0
FOR each stop
    IF alighting > passengers: report invalid event
    ELSE passengers = passengers - alighting + boarding
    IF passengers > capacity: flag capacity violation
    DISPLAY passengers at stop
```

Python source code

```
capacity = 40
stops = [
    ("Central", 18, 0),
    ("Market", 15, 5),
    ("College", 12, 8),
    ("Depot", 4, 10),
]
passengers = 0
violations = []

for stop, boarding, alighting in stops:
    if alighting > passengers:
        print(stop, "INVALID: alighting exceeds passengers onboard")
        continue
    passengers = passengers - alighting + boarding
    status = "OK"
    if passengers > capacity:
        status = "CAPACITY VIOLATION"
        violations.append(stop)
    print(stop, "onboard =", passengers, status)

print("Final passengers:", passengers)
if violations:
    print("Capacity exceeded at:", ", ".join(violations))
else:
    print("No capacity violations")
```

Test case(s)

Capacity is 40. The sequence of boarding/alighting events results in 18, 28, 32 and 26 passengers.

Sample output (screenshot-ready text)

```
Central onboard = 18 OK
Market onboard = 28 OK
College onboard = 32 OK
Depot onboard = 26 OK
Final passengers: 26
No capacity violations
```

Conclusion

The running count provides an easy-to-check simulation of occupancy at every stop and flags capacity issues.

7. Nested List Examination Seating Validation

Problem statement

Validate a seating matrix for duplicate roll numbers, reserved seats, empty positions and invalid student allocations.

Objective

Check a matrix using nested loops and lists, producing a clear issue report.

Assumptions

• 'R' means reserved, None means empty, and positive integers are student roll numbers.

• Only roll numbers in the registered list are valid.

• A reserved position must contain 'R' and cannot contain a student.

Algorithm

1. Traverse every row and seat.

2. Check reserved positions and invalid values.

3. Record roll-number positions.

4. Report duplicates and empty seats.

5. Print a summary.

Flowchart (text form)

Start → Read matrix and registered rolls → Traverse each seat → Reserved/empty/invalid? → Record issue → Count roll occurrences → Report duplicates → Print summary → Stop.

Pseudocode

```
positions = empty list
FOR each row and seat position
    IF seat is None: report empty
    ELSE IF seat is not 'R':
        IF seat not registered: report invalid roll
        ELSE record roll and location
FOR each recorded roll
    IF occurrence count > 1: report duplicate
DISPLAY all issues
```

Python source code

```
registered = [201, 202, 203, 204, 205]
reserved_positions = [(0, 1)]
seating = [
    [201, "R", 202],
    [203, 202, None],
    [999, 204, 205],
]

locations = {}
issues = []
for r, row in enumerate(seating):
    for c, seat in enumerate(row):
        if (r, c) in reserved_positions:
            if seat != "R":
                issues.append("Reserved seat overwritten at " + str((r, c)))
            continue
        if seat is None:
            issues.append("Empty seat at " + str((r, c)))
        elif seat == "R":
            issues.append("Unexpected reserved marker at " + str((r, c)))
        elif seat not in registered:
            issues.append("Invalid roll " + str(seat) + " at " + str((r, c)))
        else:
            if seat not in locations:
                locations[seat] = []
            locations[seat].append((r, c))

for roll, places in locations.items():
    if len(places) > 1:
        issues.append("Duplicate roll " + str(roll) + " at " + str(places))
```

```
print("Validation report")
if issues:
    for issue in issues:
        print("-", issue)
else:
    print("No issues found")
print("Registered rolls seated:", len(locations))
print("Empty seats:", sum(row.count(None) for row in seating))
```

Test case(s)

The matrix repeats roll 202, contains one empty seat, and contains unregistered roll 999.

Sample output (screenshot-ready text)

```
Validation report
- Empty seat at (1, 2)
- Invalid roll 999 at (2, 0)
- Duplicate roll 202 at [(0, 2), (1, 1)]
Registered rolls seated: 4
Empty seats: 1
```

Conclusion

The validator detects common seating errors without changing the original matrix, making it suitable for pre-exam checks.

8. Integrated University Fee and Eligibility Audit

Problem statement

Combine fee balance, attendance, internal marks and examination eligibility data to identify students with outstanding conditions and report reasons.

Objective

Integrate related student records using tuples, lists and conditional checks.

Assumptions

• Each student tuple is (roll, name, total_fee, paid_fee, attendance_percent, internal_mark).

• Exam requirements: no outstanding fee, attendance >= 75%, internal mark >= 40/100.

• All amounts are in rupees and paid fee cannot be negative.

Algorithm

1. Read each student's record.

2. Calculate fee balance.

3. Check each eligibility condition independently.

4. Store every reason that applies.

5. Report eligible students and students with outstanding conditions.

Flowchart (text form)

Start → Read student record → Calculate balance → Check fee, attendance and internal marks → Collect reasons → Any reason? → Report pending conditions or eligible → Next student → Stop.

Pseudocode

```
FOR each student
    balance = total_fee - paid_fee
    reasons = empty list
    IF balance > 0: add fee due
    IF attendance < 75: add attendance shortage
    IF internal_mark < 40: add internal shortage
    IF reasons empty: print ELIGIBLE
    ELSE print PENDING with all reasons
```

Python source code

```
students = [
    (301, "Anu", 50000, 50000, 88, 72),
    (302, "Bharat", 50000, 42000, 80, 55),
    (303, "Chitra", 50000, 50000, 68, 35),
    (304, "Deepak", 50000, 50000, 79, 40),
]

for roll, name, total_fee, paid_fee, attendance, internal in students:
    balance = total_fee - paid_fee
    reasons = []
    if balance > 0:
        reasons.append("fee due Rs." + str(balance))
    if attendance < 75:
        reasons.append("attendance below 75%")
    if internal < 40:
        reasons.append("internal mark below 40")
    if reasons:
        print(roll, name, "PENDING:", "; ".join(reasons))
    else:
        print(roll, name, "ELIGIBLE for examination")
```

Test case(s)

Four student records include a fee balance, low attendance, low internal marks and one fully eligible student.

Sample output (screenshot-ready text)

```
301 Anu ELIGIBLE for examination
302 Bharat PENDING: fee due Rs.8000
303 Chitra PENDING: attendance below 75%; internal mark below 40
```

```
304 Deepak ELIGIBLE for examination
```

Conclusion

The audit checks each condition separately and reports all outstanding reasons instead of stopping at the first failure.

9. Campus Event Seating and Entry Validation

Problem statement

Generate event seating assignments from nested lists, validate tickets against registration IDs, detect duplicate entries and report available seating capacity.

Objective

Apply nested-list traversal, membership checking and duplicate detection to event entry.

Assumptions

• Seats are stored as nested lists; None means available and 'R' means reserved.

• Registration IDs are unique strings.

• A valid ticket may enter once only.

• Reserved seats are not counted as available.

Algorithm

1. Build the seating layout and registration list.

2. Assign registered attendees to available seats.

3. Process ticket scans.

4. Reject unknown tickets and repeated entries.

5. Count available seats and print the entry report.

Flowchart (text form)

Start → Create seating and registration data → Assign seats → Read ticket scan → Registered? → Already entered? → Accept or reject → Count available seats → Display report → Stop.

Pseudocode

```
FOR each registration
    find first None seat and assign registration ID
entered = empty list
FOR each scanned ticket
    IF ticket not registered: reject
    ELSE IF ticket in entered: reject duplicate
    ELSE add ticket to entered; accept
Count seats equal to None
DISPLAY entry report and available capacity
```

Python source code

```
seats = [
    [None, "R", None],
    [None, None, None],
]
registrations = ["EV101", "EV102", "EV103", "EV104"]
scans = ["EV101", "EV102", "EV101", "EV999", "EV104"]

# Assign registrations to available seats
for registration in registrations:
    placed = False
    for row in seats:
        for i in range(len(row)):
            if row[i] is None:
                row[i] = registration
                placed = True
                break
        if placed:
            break
    if not placed:
        print("No seat available for", registration)

entered = []
for ticket in scans:
    if ticket not in registrations:
        print(ticket, "REJECTED: not registered")
    elif ticket in entered:
        print(ticket, "REJECTED: duplicate entry")
```

```
    else:
        entered.append(ticket)
        print(ticket, "ENTRY ACCEPTED")

available = 0
for row in seats:
    available += row.count(None)
print("Seating layout:", seats)
print("Accepted unique entries:", len(entered))
print("Available seats:", available)
```

Test case(s)

Four registrations are assigned into five non-reserved seats. Scans include one duplicate ticket and one unknown ID.

Sample output (screenshot-ready text)

```
EV101 ENTRY ACCEPTED
EV102 ENTRY ACCEPTED
EV101 REJECTED: duplicate entry
EV999 REJECTED: not registered
EV104 ENTRY ACCEPTED
Seating layout: [['EV101', 'R', 'EV102'], ['EV103', 'EV104', None]]
Accepted unique entries: 3
Available seats: 1
```

Conclusion

Registration validation and duplicate-entry checks help prevent invalid or repeated event entry while showing remaining seating capacity.

10. Advanced String-Based Registration Record Analyzer

Problem statement

Parse structured registration strings, validate field lengths and formats, extract components, detect duplicates and classify errors.

Objective

Use string operations, splitting, loops and conditions to validate records without external libraries.

Assumptions

• Format: ROLL|NAME|EMAIL|PHONE.

• Roll: exactly 6 digits; name: 2–30 alphabetic characters/spaces; email must contain one '@' and a dot after it; phone: exactly 10 digits.

• A roll number must be unique. This is a simple classroom validator, not a complete email-standard validator.

Algorithm

1. Split each record using '|'.

2. Check field count and trim whitespace.

3. Validate each field's format and length.

4. Detect duplicate roll numbers among valid-format records.

5. Classify each record as VALID or with listed errors.

Flowchart (text form)

Start → Read record string → Split into fields → Correct number of fields? → Validate each field → Check duplicate roll → Print VALID or error list → Next record → Stop.

Pseudocode

```
seen_rolls = empty list
FOR each record
    fields = split record by '|'
    IF field count != 4: report FIELD_COUNT_ERROR
    ELSE extract roll, name, email, phone
         validate roll length/digits
         validate name length/characters
         validate email basic structure
         validate phone length/digits
         IF roll already seen: add DUPLICATE_ROLL
         ELSE add roll to seen list
         print VALID if no errors, otherwise errors
```

Python source code

```
records = [
    "123456|Amar Shaik|amar@example.com|9876543210",
    "123457|Asifa|asifa@example.com|9123456780",
    "123456|Ravi|ravi@example.com|9988776655",
    "12A456|Kiran|kiranexample.com|12345",
    "123458|M|m@example.com|9876543210",
]
seen_rolls = []

for record in records:
    fields = record.split("|")
    if len(fields) != 4:
        print("INVALID:", record, "FIELD_COUNT_ERROR")
        continue

    roll, name, email, phone = [field.strip() for field in fields]
    errors = []
    if len(roll) != 6 or not roll.isdigit():
        errors.append("ROLL_FORMAT")
    if not (2 <= len(name) <= 30) or not all(
        part.isalpha() for part in name.split()
    ):
        errors.append("NAME_FORMAT")
    if email.count("@") != 1:
        errors.append("EMAIL_FORMAT")
```

```
else:
    local, domain = email.split("@")
    if not local or "." not in domain or domain.startswith(".") or domain.endswith("."):
        errors.append("EMAIL_FORMAT")
if len(phone) != 10 or not phone.isdigit():
    errors.append("PHONE_FORMAT")
if roll in seen_rolls:
    errors.append("DUPLICATE_ROLL")
else:
    seen_rolls.append(roll)

if errors:
    print("INVALID:", roll, "->", ", ".join(errors))
else:
    print("VALID:", roll, name, email, phone)
```

Test case(s)

The test records include valid entries, a duplicate roll, an invalid roll and phone, an invalid email, and a one-character name.

Sample output (screenshot-ready text)

```
VALID: 123456 Amar Shaik amar@example.com 9876543210
VALID: 123457 Asifa asifa@example.com 9123456780
INVALID: 123456 -> DUPLICATE_ROLL
INVALID: 12A456 -> ROLL_FORMAT, EMAIL_FORMAT, PHONE_FORMAT
INVALID: 123458 -> NAME_FORMAT
```

Conclusion

The analyzer uses simple string rules to extract fields, identify duplicates and explain errors. Real systems should use stronger validation and privacy protections.